Тема 7

• оператор-функция със специален синтаксис

→ унарни оператори (с 1 аргумент) – !, ++, ~

→ бинарни (2 аргумента) – +, –, *, /

→ тернарен – <bool> ? a : b

Характеризират се с:

1) приоритет

2) асоциативност

лява: +, –, *, /

дясна: =

3) позиция

префиксен: ++a, !a

постфиксен: a++

инфиксен: a + b

• предефиниране на оператори (operator overloading)

→ могат да се предефинират като външни и вътрешни функции

→ вътрешни – когато променят обекта (+=, –=, *=, /=)

→ външни – когато не го променят (+, –, *, /)

```
A operator+(const A& a1, const A& a2){...}
:
int main(){
  A a1; A a2;
  a1+=a2; → извиква се от левия обект
}
```

→ оператор ++

префиксен - връща референция към обекта A& operator++();

суфиксен - връща копие и има dummy параметър

A operator++(int) {...}

→ при OP overloading трябва да се спазват обещаните стандарти.

Пример: + не променя аргументите си

== връща bool

⋮

! - операторите =, [], (), → трябва да са член-функции

function call

- операторите ?:, ::, . не могат да се предефинират
- не могат да се създават нови оператори
- асоциативност, приоритет и позиция не могат да се предефинират
- ако предефинираме && или ||, губим предварителното изчисление
- → трябва да връща копие/указател/референция
- предефинираме [] два пъти - за константен и неконстантен достъп

приятелски класове и функции

→ имат достъп до всички данни на класа, в който са декларирани

→ често предефинираме операторите >> и << като friend функции

→ не се предава м1у класовете

```
class X {friend class Y;}
class Y {friend class Z;}
```
→ Z не е friend на X

7A) · Пример за реализация на комплексно число

.h
```
class Complex {
    public:
        Complex(double real, double im);
        double getReal() const;
        double getIm() const;
        void setReal(double real);
        void setIm(double);

        Complex getConjugated() const;
        Complex& op+=(const Complex&);
        Complex& op-=(const Complex&);
        Complex& op*=(const Complex&);
        Complex& op/=(const Complex&);

        friend std::ostream& op<<(std::ostream&, const Complex&);
        friend std::istream& op>>(std::istream&, Complex&);
```